COMPSCI 572.01 (NLP)
Pretraining 2

Slides inspired by John Hewitt at Stanford CS224N

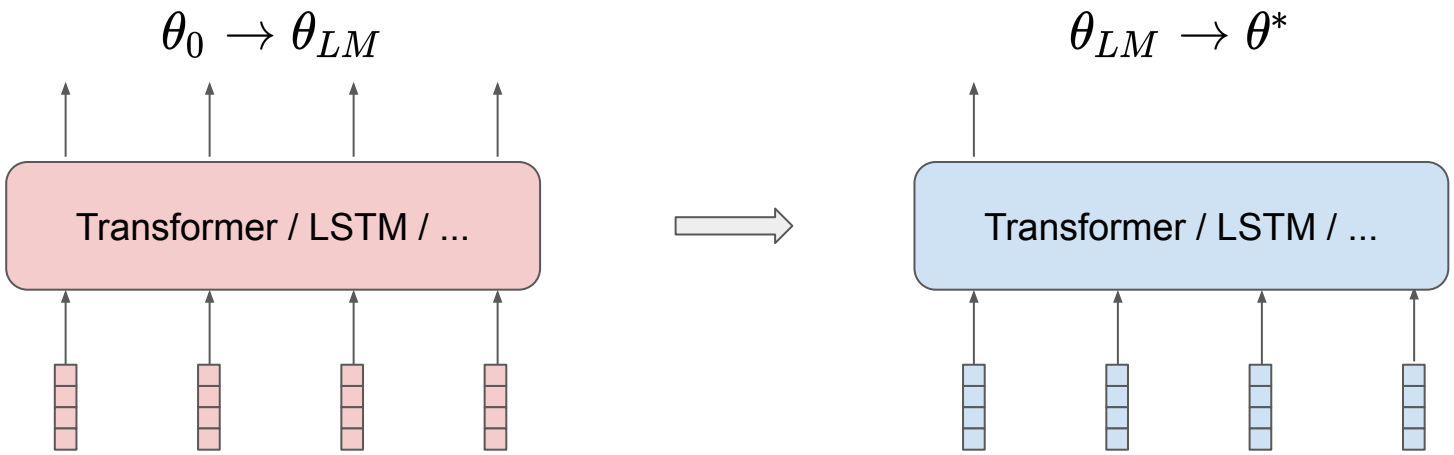
Announcements

- Project teams are due next Wednesday, March 25th by 11:59 PM
  - See Ed for details
  - Strongly encouraged to form teams of 2 or 3 students
  - If you don’t have a team we can help you find one

Overview

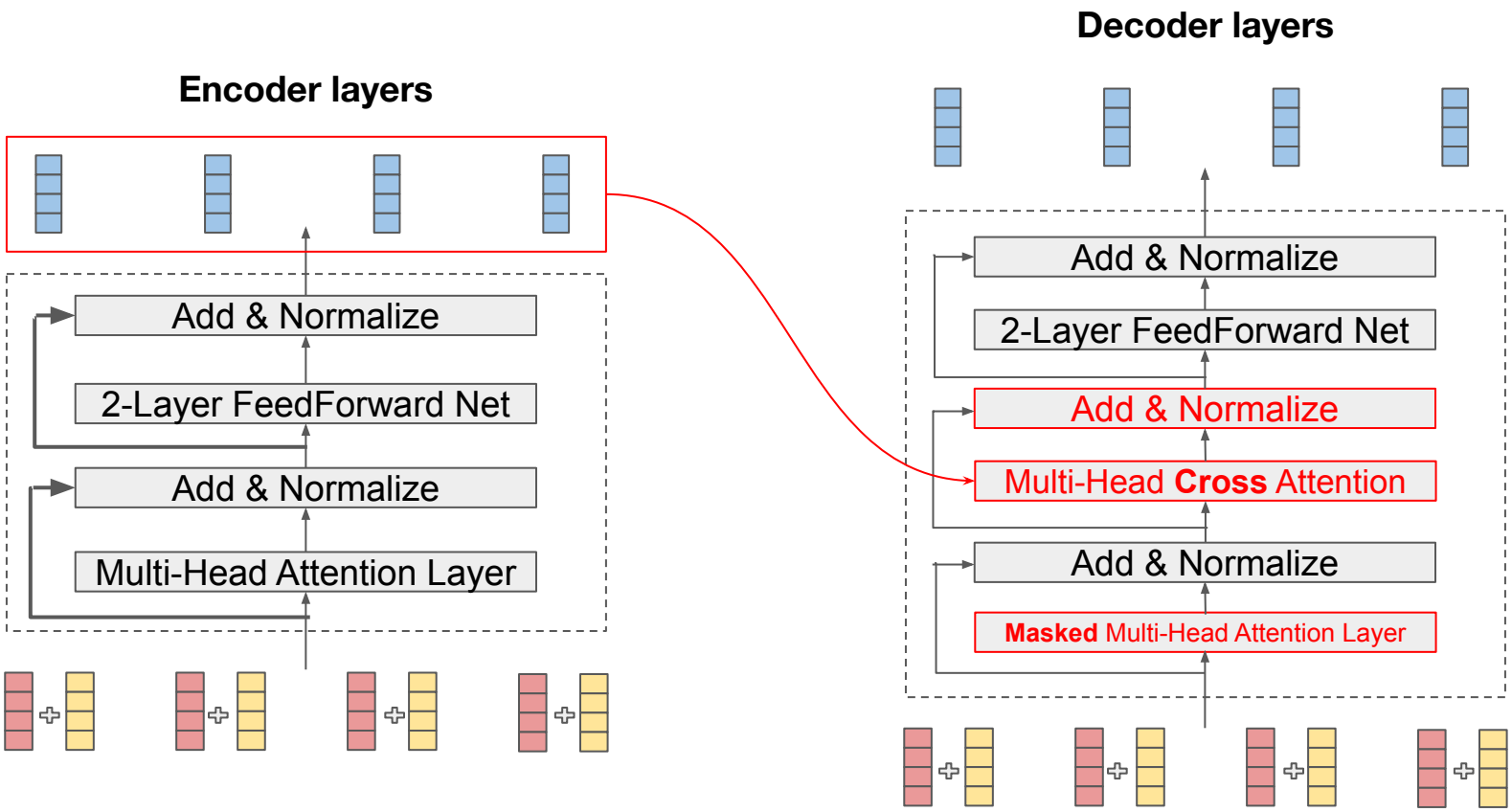
- Pretrained model architectures
  - Encoder only (BERT)
  - Decoder only (GPT)
  - Encoder-Decoder (T5)
- Multilingual models

Recap: Pretraining Recipe

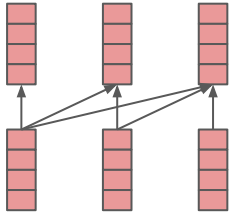


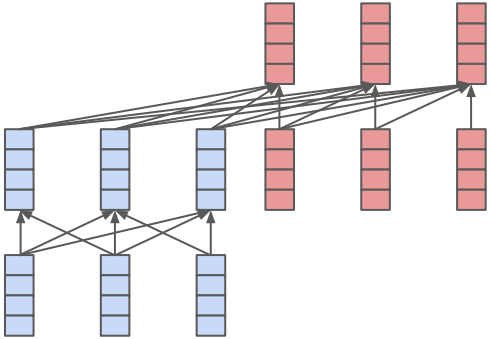
1. **Pretrain** the network on language modeling or some other reconstruction objective over unlabeled data
2. **Finetune** the network on task-specific labeled data

Recap: Transformer Encoder-Decoders



Recap: Model architectures for pretraining





Recap: BERT

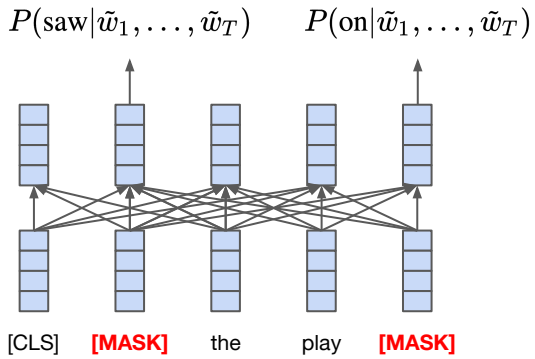
- Encoder models receive **bidirectional context**
  - Hence we cannot use language modeling to train

- Instead, use **masked language modeling**
  - Replace a fraction of input tokens with a special “[MASK]” token and predict them from the output

$$h_1, \ldots, h_T = \text{Encoder}(\tilde{w}_1, \ldots, \tilde{w}_T)$$

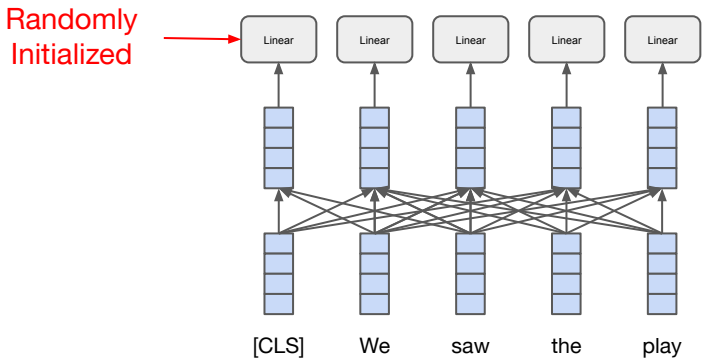
$$P(w_i | \tilde{w}_1, \ldots, \tilde{w}_T) = \text{softmax}(Wh_i + b)$$

$$\text{loss} = -\sum_{i:\tilde{w}_i=[\text{MASK}]} \log P(w_i | \tilde{w}_1, \ldots, \tilde{w}_T)$$



Recap: Finetuning BERT

- Classification
  - Add a linear layer on top of [CLS] representation
- Sequence labeling
  - Add a linear layer on top of each token's representation
  - Works well even without CRF!
- Generation
  - No obvious way



Devlin et al; 2018

BERT Results

System	MNLI-(m/mm)	QQP	QNLI	SST-2	CoLA	STS-B	MRPC	RTE	**Average**
	392k	363k	108k	67k	8.5k	5.7k	3.5k	2.5k	-
Pre-OpenAI SOTA	80.6/80.1	66.1	82.3	93.2	35.0	81.0	86.0	61.7	74.0
BiLSTM+ELMo+Attn	76.4/76.1	64.8	79.8	90.4	36.0	73.3	84.9	56.8	71.0
OpenAI GPT	82.1/81.4	70.3	87.4	91.3	45.4	80.0	82.3	56.0	75.1
$\text{BERT}_{\text{BASE}}$	84.6/83.4	71.2	90.5	93.5	52.1	85.8	88.9	66.4	79.6
$\text{BERT}_{\text{LARGE}}$	**86.7/85.9**	**72.1**	**92.7**	**94.9**	**60.5**	**86.5**	**89.3**	**70.1**	**82.1**

With pretrained models, larger sizes are almost always better

Devlin et al; 2018

RoBERTa: Robustly Optimized BERT

- Train for longer with much larger batch sizes
  - And remove next sentence prediction
- Add more data

Model	data	bsz	steps	SQuAD (v1.1/2.0)	MNLI-m	SST-2
RoBERTa						
with BOOKS + WIKI	16GB	8K	100K	93.6/87.3	89.0	95.3
+ additional data (§3.2)	160GB	8K	100K	94.0/87.7	89.3	95.6
+ pretrain longer	160GB	8K	300K	94.4/88.7	90.0	96.1
+ pretrain even longer	160GB	8K	500K	**94.6/89.4**	**90.2**	**96.4**
$\text{BERT}_{\text{LARGE}}$						
with BOOKS + WIKI	13GB	256	1M	90.9/81.8	86.6	93.7

Liu et al; 2019

Masking variants

- Random masking
  - Take random 15% subword tokens

Di Caprio ##’s acting in Inception was inc ##red ##ible

Di [MASK] ##’s acting in Inception was inc [MASK] ##ible

Masking variants

- Random masking
  - Take random 15% subword tokens
- Whole word masking
  - Mask all subword tokens in the same word together

Di Caprio ##’s acting in Inception was inc ##red ##ible

Di Caprio ##’s acting in Inception was [MASK] [MASK] [MASK]

Masking variants

- Random masking
  - Take random 15% subword tokens
- Whole word masking
  - Mask all subword tokens in the same word together
- Span masking (SpanBERT; Joshi et al, 2019)
  - Mask contiguous multi-word spans together

Di Caprio ##’s acting in Inception was inc ##red ##ible

Di Caprio ##’s [MASK] [MASK] [MASK] was inc ##red ##ible

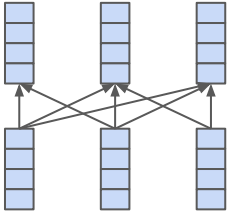
Masking variants

- Random masking
  - Take random 15% subword tokens
- Whole word masking
  - Mask all subword tokens in the same word together
- Span masking (SpanBERT; Joshi et al, 2019)
  - Mask contiguous multi-word spans together
- Salient span masking (SSM, Guu et al, 2020)
  - Mask named entities and dates

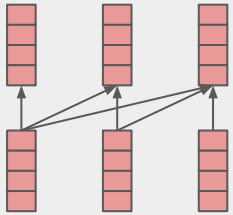
Di Caprio ##’s acting in Inception was inc ##red ##ible

[MASK] [MASK] [MASK] acting in [MASK] was inc ##red ##ible

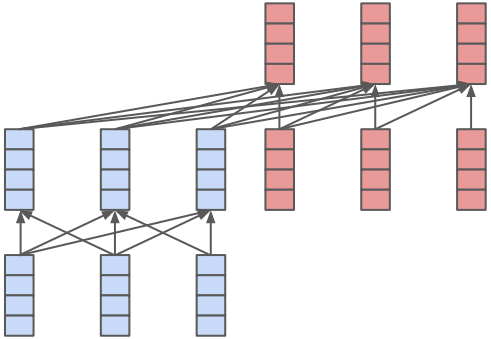
Model architectures for pretraining



Encoder only



Decoder only



Encoder-Decoders

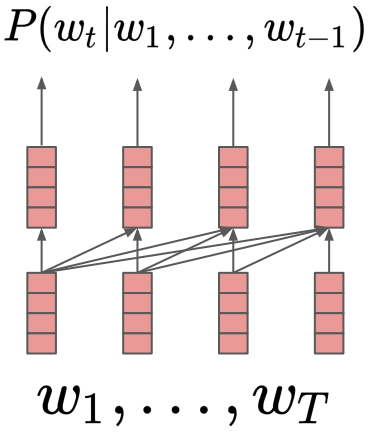
Decoder-only pretraining

- Pretrain on language modeling

$$h_1, \ldots, h_T = \text{Decoder}(w_1, \ldots, w_T)$$

$$P(w_t | w_1, \ldots, w_{t-1}) = \text{softmax}(W h_{t-1} + b)$$

$$\text{loss} = -\sum_{t=1}^{T} \log P(w_t | w_1, \ldots, w_{t-1})$$

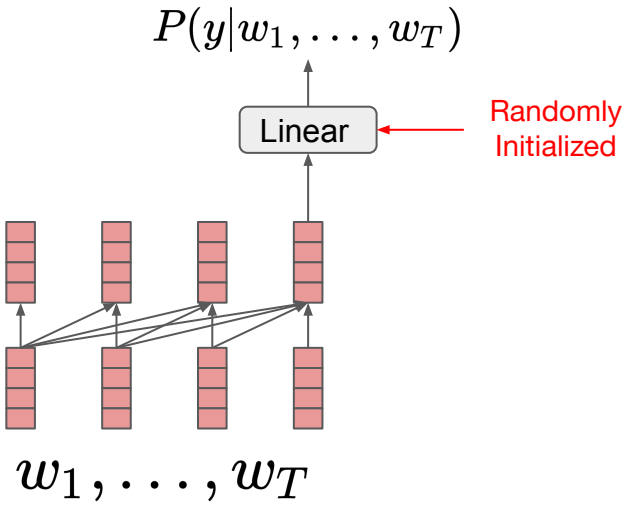


Decoder-only pretraining

- Pretrain on language modeling
- For finetuning:
  - One option is to add a linear classifier on top of last word's hidden state

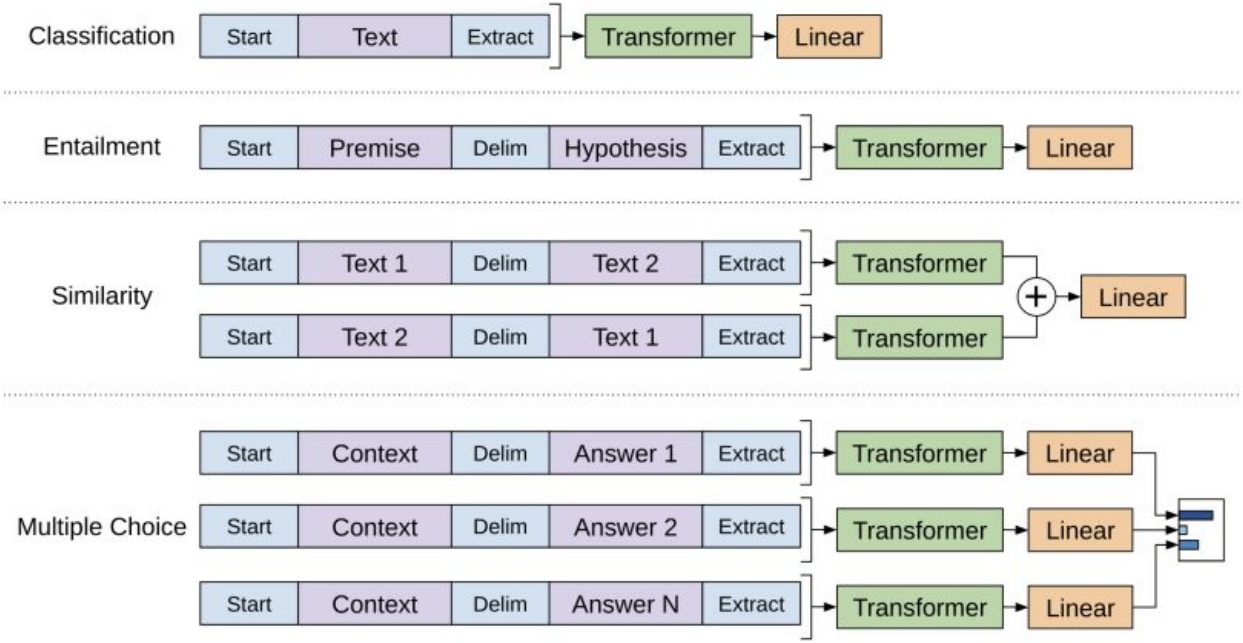
$$h_1, \ldots, h_T = \text{Decoder}(w_1, \ldots, w_T)$$

$$P(y|w_1, \ldots, w_T) = \text{softmax}(Wh_T + b)$$



GPT (1)

Classification using decoder-only models:



Generative Pretrained Transformer, Radford et al; 2018

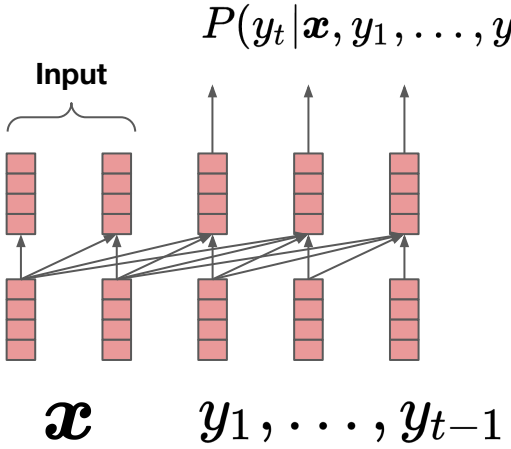
Decoder-only pretraining

Decoder-only models are also sequence-to-sequence models!

- We can simply fix the input sequence, and train as a conditional language model
- In this way we can frame both classification and generation tasks as seq2seq

$$h_1, \ldots, h_t = \text{Decoder}(\boldsymbol{x}, y_1, \ldots, y_t)$$

$$P(y_t | \boldsymbol{x}, y_1, \ldots, y_{t-1}) = \text{softmax}(Wh_t + b)$$



Everything is pretrained!

GPT (1)

- Model: 12 layers, 768-dim hidden states, 3072-dim feedforward layers, 12 attention heads
  - ~117M params

- Data: BooksCorpus (7000 books, 800M tokens)

System	MNLI-(m/mm)	QQP	QNLI	SST-2	CoLA	STS-B	MRPC	RTE	**Average**
	392k	363k	108k	67k	8.5k	5.7k	3.5k	2.5k	-
Pre-OpenAI SOTA	80.6/80.1	66.1	82.3	93.2	35.0	81.0	86.0	61.7	74.0
BiLSTM+ELMo+Attn	76.4/76.1	64.8	79.8	90.4	36.0	73.3	84.9	56.8	71.0
OpenAI GPT	82.1/81.4	70.3	87.4	91.3	45.4	80.0	82.3	56.0	75.1
$\text{BERT}_{\text{BASE}}$	84.6/83.4	71.2	90.5	93.5	52.1	85.8	88.9	66.4	79.6
$\text{BERT}_{\text{LARGE}}$	**86.7/85.9**	**72.1**	**92.7**	**94.9**	**60.5**	**86.5**	**89.3**	**70.1**	**82.1**

- Worse than BERT on classification, but can be used for generation!

Radford et al; 2018

GPT-2

- Model: 1.5B params, 48 layers, 1600-dim hidden states
- Data: WebText (outbound links from Reddit), ~10B tokens of text
- First model to produce convincing samples of natural language:

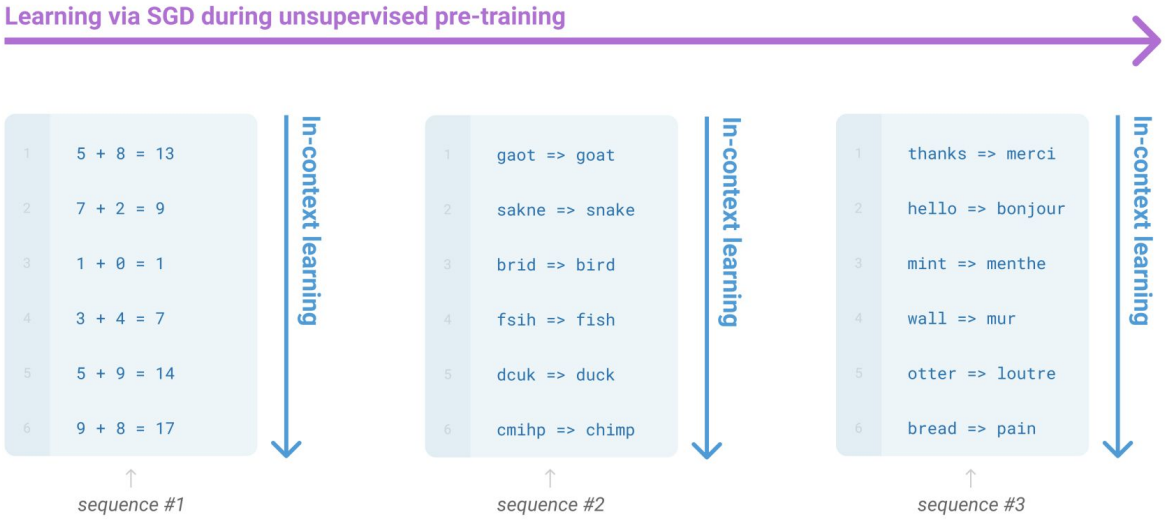
Context (human-written): In a shocking finding, scientist discovered a herd of unicorns living in a remote, previously unexplored valley, in the Andes Mountains. Even more surprising to the researchers was the fact that the unicorns spoke perfect English.
GPT-2: The scientist named the population, after their distinctive horn, Ovid's Unicorn. These four-horned, silver-white unicorns were previously unknown to science. Now, after almost two centuries, the mystery of what sparked this odd phenomenon is finally solved. Dr. Jorge Pérez, an evolutionary biologist from the University of La Paz, and several companions, were exploring the Andes Mountains when they found a small valley, with no other animals or humans. Pérez noticed that the valley had what appeared to be a natural fountain, surrounded by two peaks of rock and silver snow.

Language Models are Unsupervised Multitask Learners (OpenAI)

GPT-3

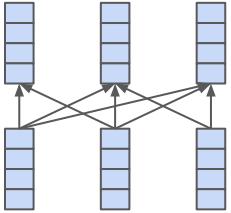
- Model: 175B params, 96 layers, 96 heads, 12K-dim hidden states
- Data: 500B tokens of text (Common Crawl + WebText + Wikipedia + Books)
- First model to show in-context learning (without gradient steps)

Not publicly available :(

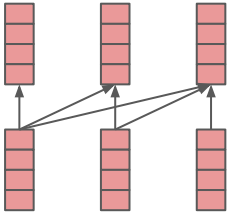


Language models are few-shot learners (May, 2020)

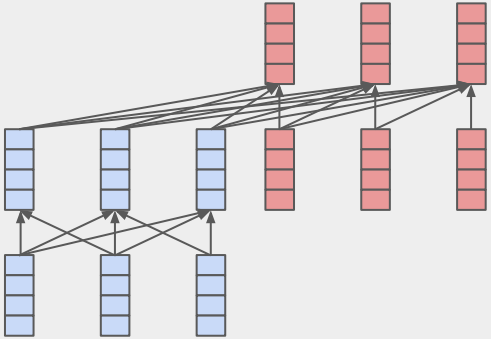
Model architectures for pretraining



Encoder only



Decoder only



Encoder-Decoders

Encoder-Decoder Pretraining

We can use language modeling: For every input,

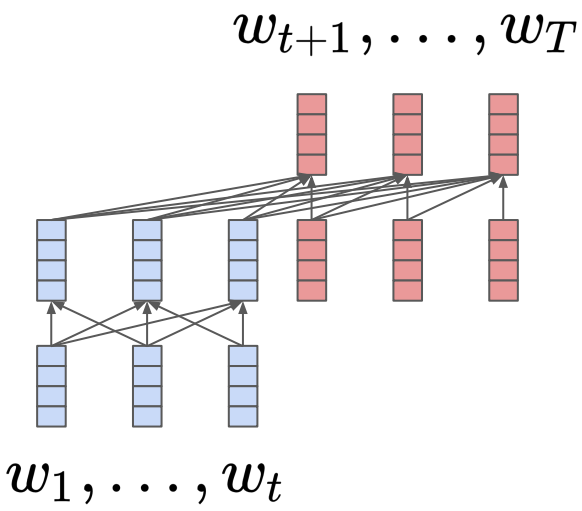
- Encode a prefix using the bidirectional encoder
- Predict the rest using the unidirectional decoder

$$h_1, \ldots, h_t = \text{Encoder}(w_1, \ldots, w_t)$$

$$h_{t+1}, \ldots, h_T = \text{Decoder}(w_{t+1}, \ldots, w_T, h_1, \ldots, h_t)$$

$$P(w_{t'} | w_{<t'}) = \text{softmax}(W h_{t'} + b)$$

$$\text{loss} = -\sum_{t'=t+1}^{T} \log P(w_{t'} | w_{<t'})$$



Exploring the limits of text-to-text transfer learning Raffel et al; 2019

Encoder-Decoder Pretraining

Or we can again use *masked* language modeling (a.k.a *Denoising*)

- Replace spans in the input with special tokens
- Predict those spans at the output

Original text

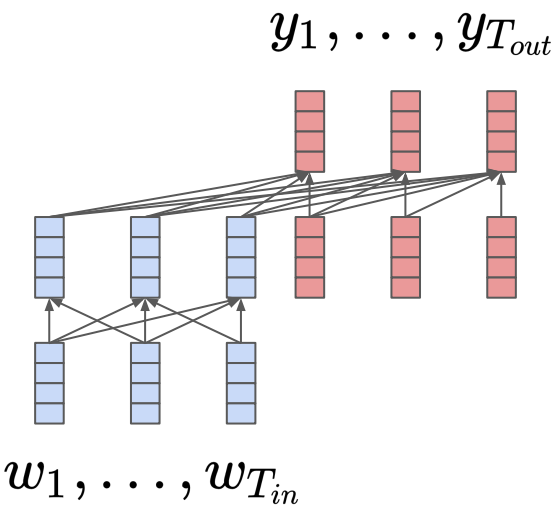
Thank you ~~for inviting~~ me to your party ~~last~~ week.

Inputs

Thank you <X> me to your party <Y> week.

Targets

<X> for inviting <Y> last <Z>

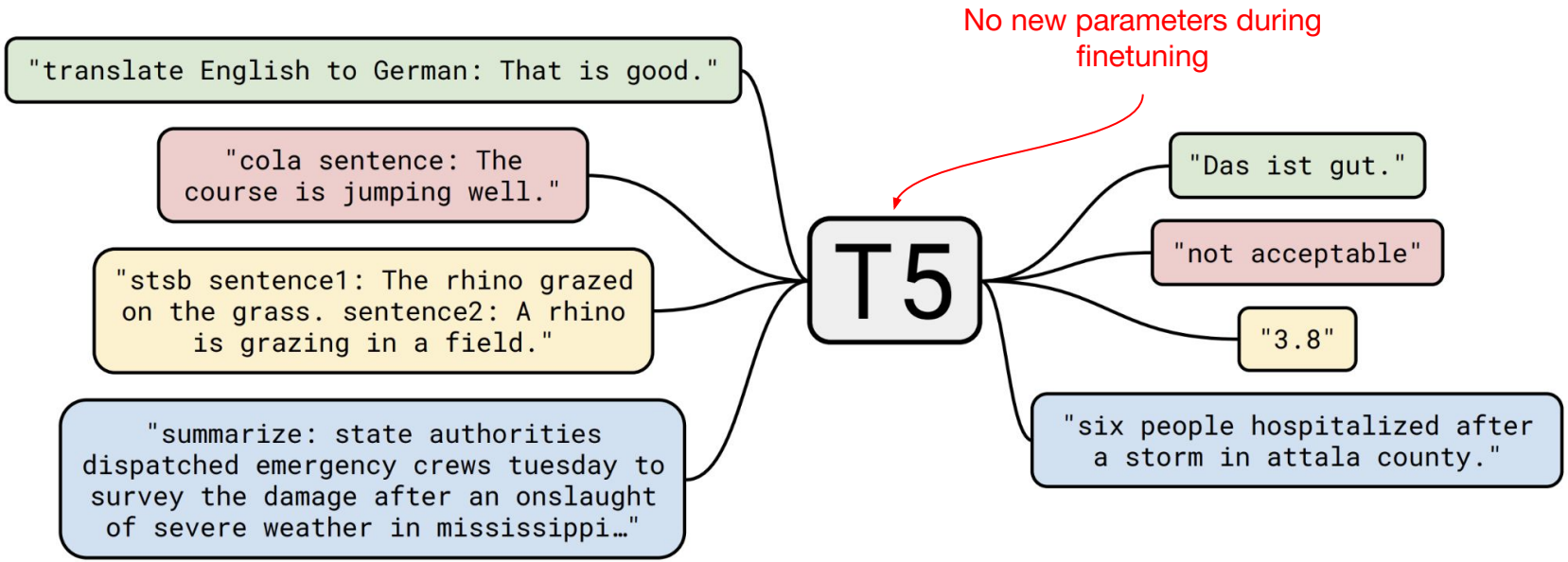


Exploring the limits of text-to-text transfer learning Raffel et al; 2019

Finetuning

Text-To-Text Transfer Transformer (T5)

- Formulate all tasks as a sequence-to-sequence problem

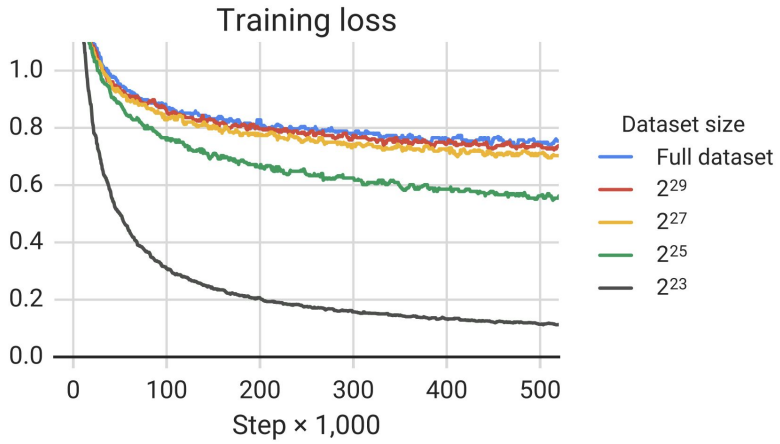


Exploring the limits of text-to-text transfer learning Raffel et al; 2019

Pretraining Data

- C4 corpus: 745gb of *preprocessed* text extracted from Common Crawl (~150B tokens)

- Model starts memorizing smaller datasets, hurting downstream performance



A benchmark collection of NLP tasks

Number of tokens	Repeats	GLUE
★ Full data set	0	**83.28**
2^{29}	64	**82.87**
2^{27}	256	82.62
2^{25}	1,024	79.55
2^{23}	4,096	76.34

Exploring the limits of text-to-text transfer learning Raffel et al; 2019

Pretraining objective

Architecture	Objective	Params	Cost	GLUE
★ Encoder-decoder	Denoising	$2P$	M	**83.28**
Enc-dec, shared	Denoising	P	M	82.81
Enc-dec, 6 layers	Denoising	P	$M/2$	80.88
Language model	Denoising	P	M	74.70
Prefix LM	Denoising	P	M	81.82
Encoder-decoder	LM	$2P$	M	79.56
Enc-dec, shared	LM	P	M	79.60
Enc-dec, 6 layers	LM	P	$M/2$	78.67
Language model	LM	P	M	73.78
Prefix LM	LM	P	M	79.68

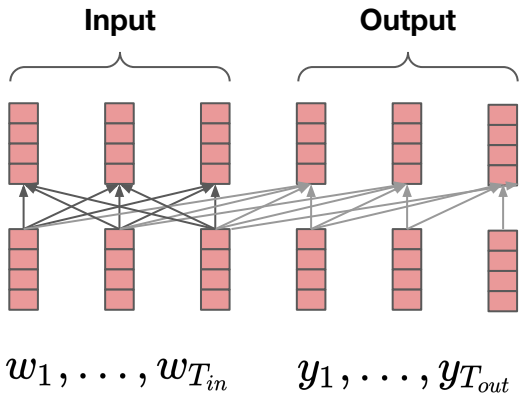
Denoising (masked LM) works better than pure LM

Exploring the limits of text-to-text transfer learning Raffel et al; 2018

Model architecture

Having separate encoders and decoders works best

Architecture	Objective	Params	Cost	GLUE
★ Encoder-decoder	Denoising	$2P$	M	**83.28**
Enc-dec, shared	Denoising	P	M	82.81
Enc-dec, 6 layers	Denoising	P	$M/2$	80.88
Language model	Denoising	P	M	74.70
Prefix LM	Denoising	P	M	81.82
Encoder-decoder	LM	$2P$	M	79.56
Enc-dec, shared	LM	P	M	79.60
Enc-dec, 6 layers	LM	P	$M/2$	78.67
Language model	LM	P	M	73.78
Prefix LM	LM	P	M	79.68



Exploring the limits of text-to-text transfer learning Raffel et al; 2019

Text-to-Text Transfer Transformer (T5)

- Five model sizes
  - T5-11B: 24 layers, 1024-dim hidden states, 65K-dim feedforward states, 128 attention heads

Exploring the limits of text-to-text transfer learning Raffel et al; 2019

Text-to-Text Transfer Transformer (T5)

- Five model sizes
  - T5-11B: 24 layers, 1024-dim hidden states, 65K-dim feedforward states, 128 attention heads
- On classification tasks:

Model	GLUE Average
Previous best	89.4[a]
T5-Small (60M)	77.4
T5-Base (220M)	82.7
T5-Large (770M)	86.4
T5-3B	88.5
T5-11B	**90.3**

BERT-base (110M)	79.6
BERT-large (340M)	82.1

Exploring the limits of text-to-text transfer learning Raffel et al; 2019

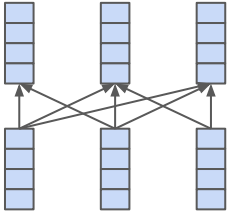
Text-to-Text Transfer Transformer (T5)

- Five model sizes
  - T5-11B: 24 layers, 1024-dim hidden states, 65K-dim feedforward states, 128 attention heads
- On sequence-to-sequence tasks:

	Machine Translation			Summarization		
Model	WMT EnDe BLEU	WMT EnFr BLEU	WMT EnRo BLEU	CNN/DM ROUGE-1	CNN/DM ROUGE-2	CNN/DM ROUGE-L
Previous best	**33.8**[e]	**43.8**[e]	**38.5**[f]	43.47[g]	20.30[g]	40.63[g]
T5-Small	26.7	36.0	26.8	41.12	19.56	38.35
T5-Base	30.9	41.2	28.0	42.05	20.34	39.40
T5-Large	32.0	41.5	28.1	42.50	20.68	39.75
T5-3B	31.8	42.6	28.2	42.72	21.02	39.94
T5-11B	32.1	43.4	28.1	**43.52**	**21.55**	**40.69**

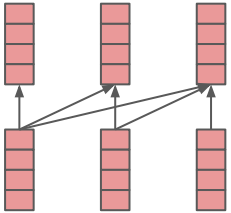
Exploring the limits of text-to-text transfer learning Raffel et al; 2019

Summary: Pretraining Model Architectures



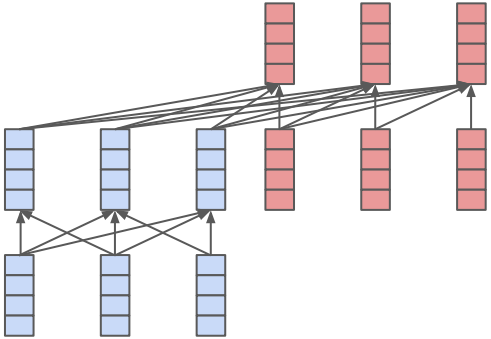
Encoder only

- Condition on both previous and future words
- Trained using masked language modeling
- Cannot be used for generating text



Decoder only

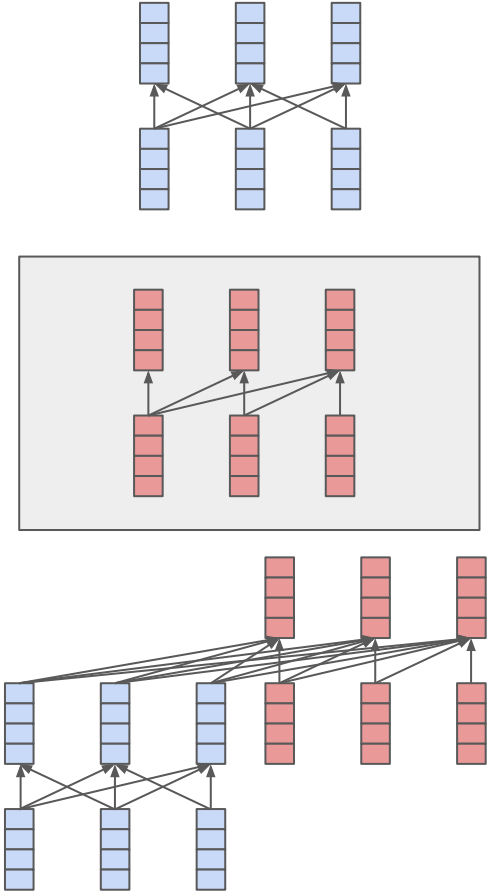
- Only condition on previous words
- Trained using language modeling
- Useful for generating text



Encoder-Decoders

- Work well for both classification and generation!
- Can be trained using language modeling or any of its variants

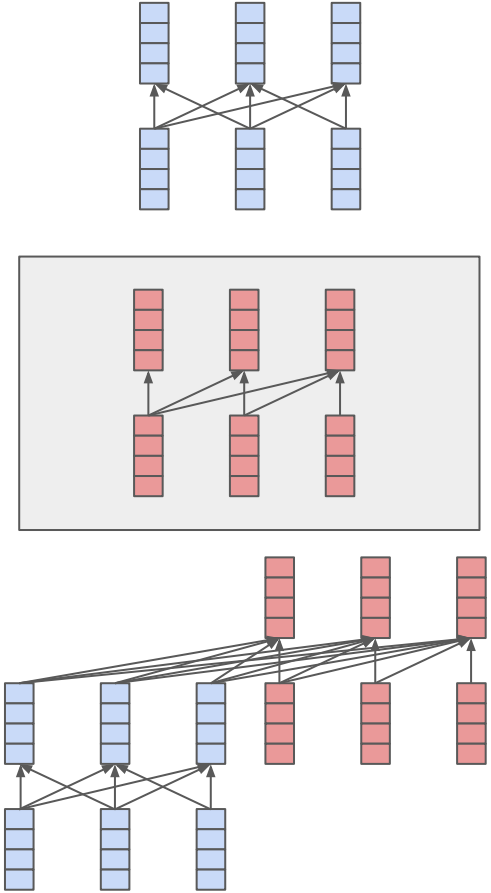
Summary: Pretraining Model Architectures



Today's LLMs are almost all exclusively **decoder-only** trained with **language modeling**

Why?

Summary: Pretraining Model Architectures

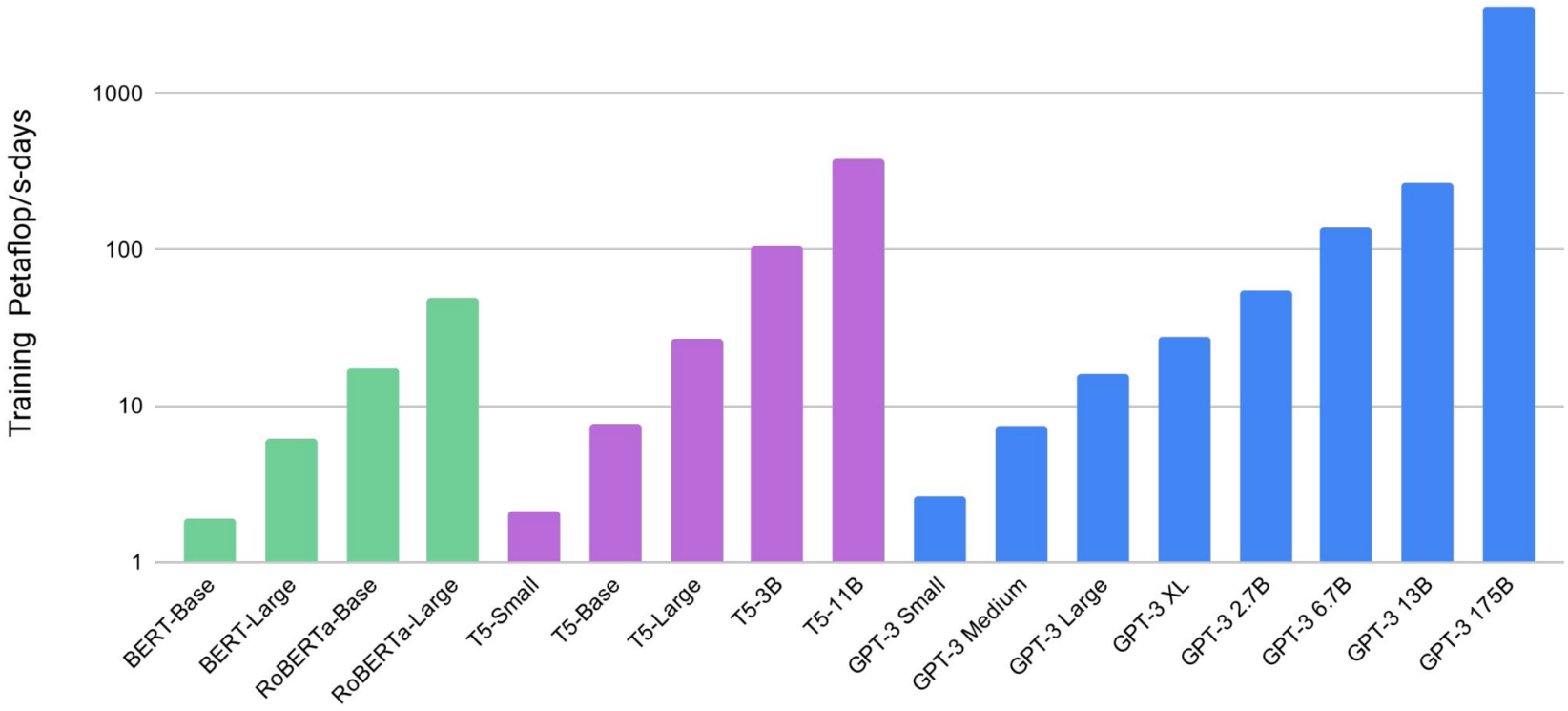


Today's LLMs are almost all exclusively **decoder-only** trained with **language modeling**

Why?

- Simplicity: don't need to keep around 2 transformers (with different architectures)
- Parameter efficiency: same set of parameters used for encoding and decoding

Training Cost



What about other languages?

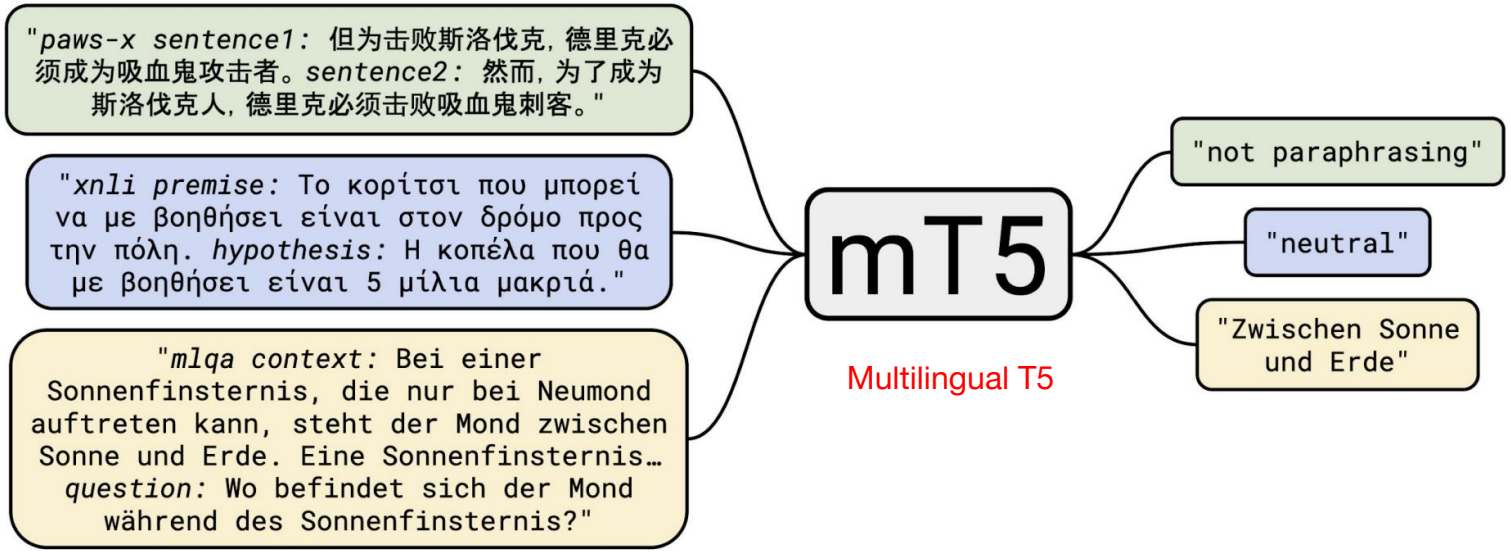


Figure credit: Colin Raffel

Recap: Byte pair encoding

- Start with a vocabulary containing all characters
- Take the most common adjacent characters in a corpus and add them to the vocabulary
  - E.g. if “a” and “a” occur together frequently add “aa” to the vocabulary
- Replace each occurrence of the pair in the corpus with the new vocabulary item and repeat the process until vocabulary reaches desired size

a a a b d a a a b a c

aa a b d aa a b a c

aa ab d aa ab a c

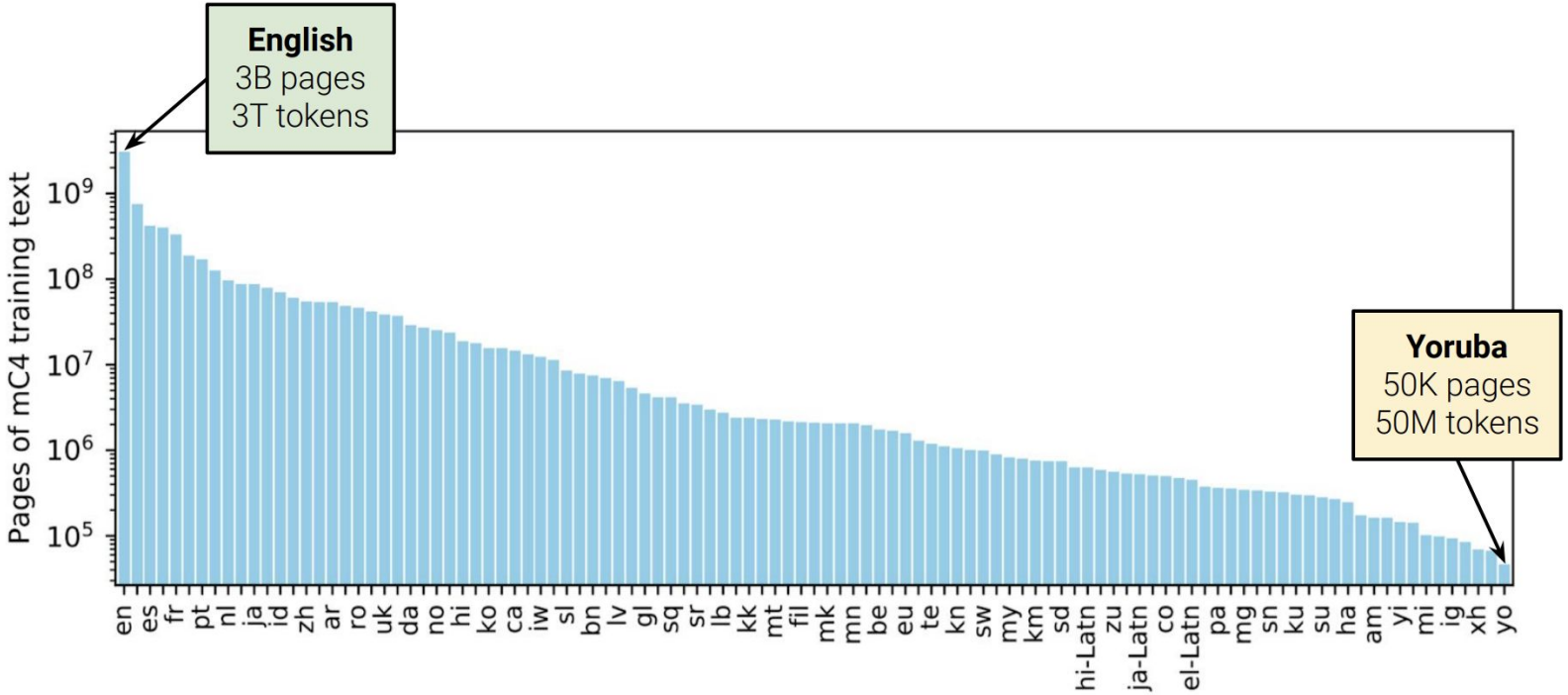
aaab d aaab a c

Vocab

a
b
c
d
aa
ab
aaab

mC4: Multilingual C4 dataset

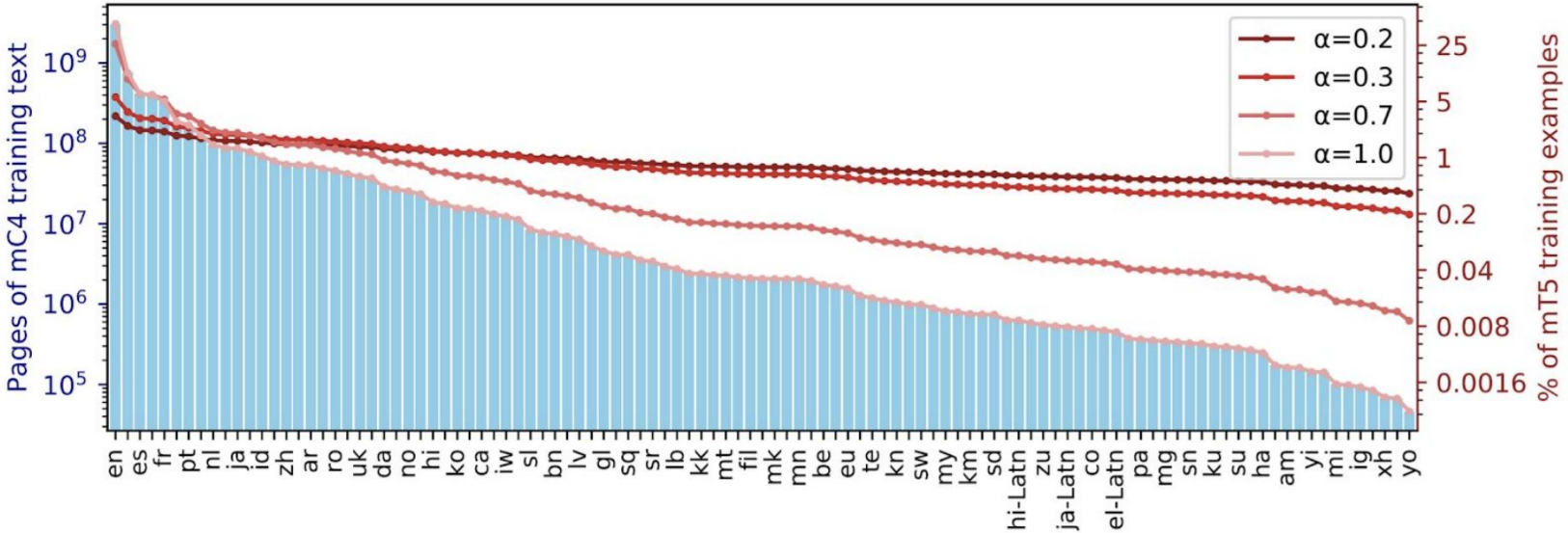
- Text in 101 languages using similar process to C4



Much larger vocabulary: 250,000 subwords

mC4: Multilingual C4 dataset

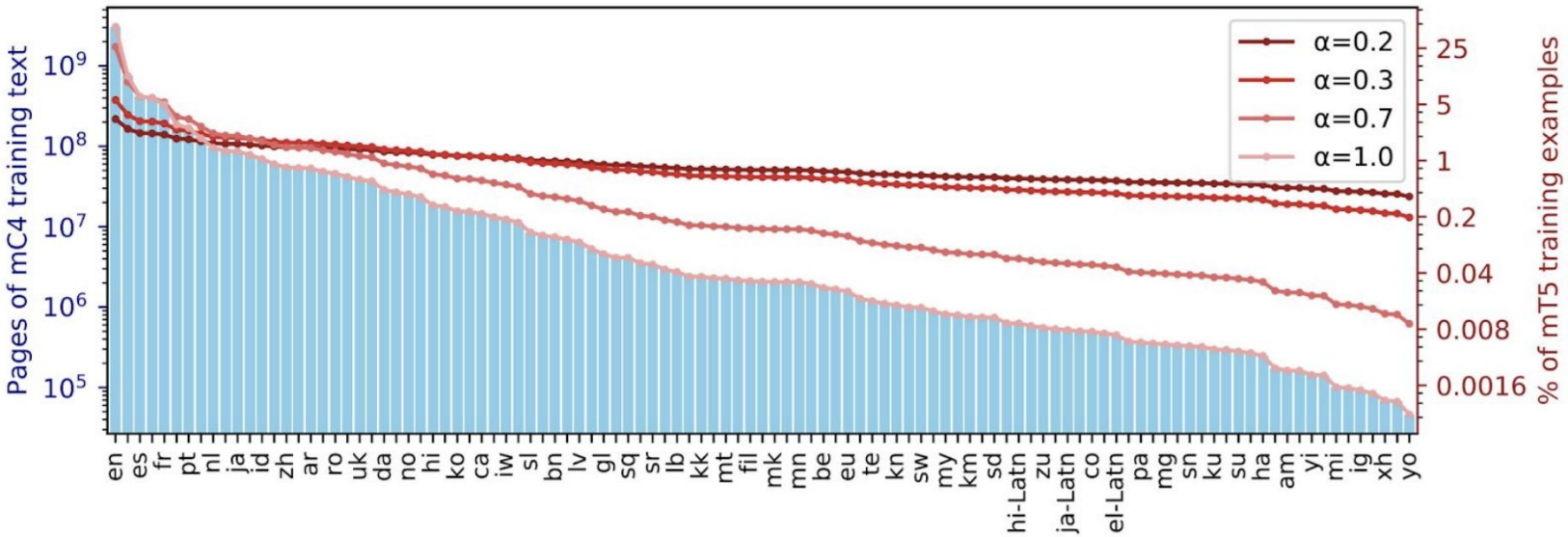
- Sample examples using $P(L) \propto |L|^{\alpha}$



mC4: Multilingual C4 dataset

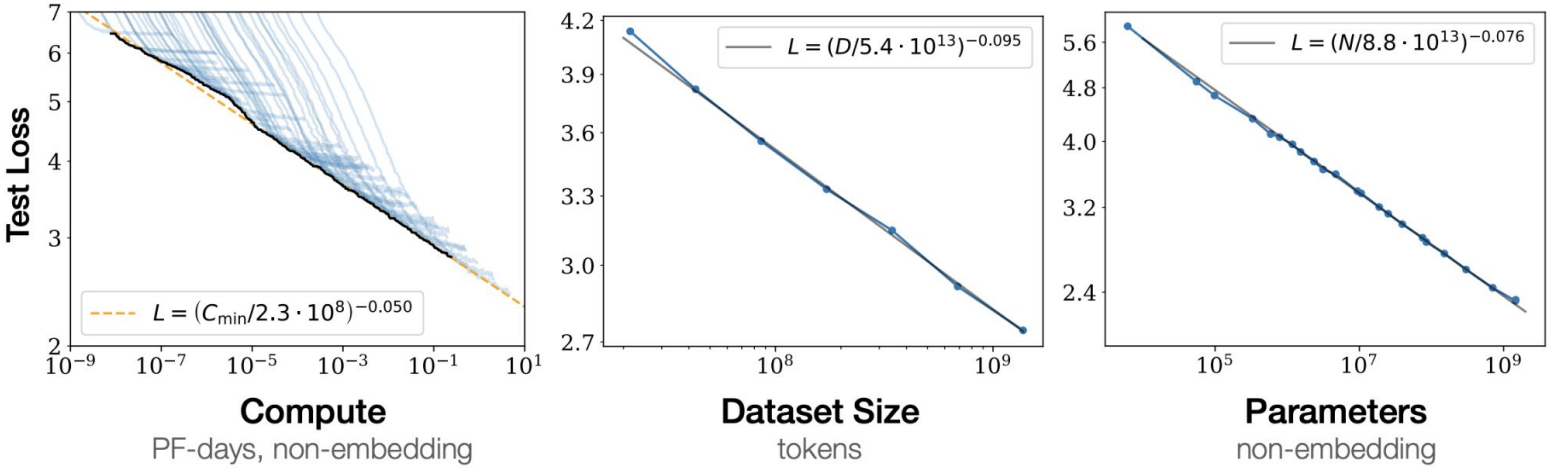
- Sample examples using $P(L) \propto |L|^{\alpha}$

XNLI Zero-shot Accuracy		
	Urdu	Russian
α=0.2	**73.9**	81.2
α=0.3	73.5	81.5
α=0.7	71.7	**82.8**



That’s it!

- Next lecture: Scaling.



Kaplan et al; 2020